

Séance 5 : Performance, monitoring et livrable final

Owner	 louis reynouard
Tags	

Contexte

Votre pipeline streaming complet tourne : ingestion Bronze, transformation Silver, qualité Gold, scoring ML, monitoring drift. Vous l'avez construit séance par séance. Cette dernière séance pose la question qui se pose systématiquement en production : est-ce que ça tient quand le volume augmente ?

1. Monitoring d'un job streaming : lire l'historique des runs

Avec la boucle `availableNow`, il n'y a pas de job continu à observer en temps réel. Chaque itération de la boucle produit un run discret qui s'arrête seul. Le monitoring se fait donc de deux façons complémentaires : dans le code via `time.time()`, et dans la Spark UI via l'historique des queries.

Mesurer la durée dans la boucle

C'est la méthode principale sur serverless. Vous encadrez `query.awaitTermination()` avec deux `time.time()` pour obtenir la durée exacte de chaque run.

```
import time

log = [] # tableau de suivi inter-itérations

for i in range(10):
    sim = TaxiStreamSimulator(output_path=SIM_OUTPUT_PATH, ra
```

```

te=RATE,
                                interval_seconds=0, mode="normal",
                                duration_batches=1, seed=i)
t = threading.Thread(target=sim.run)
t.start()
t.join()

start = time.time()
query = (df_stream.writeStream
        .format("delta")
        .outputMode("append")
        .option("checkpointLocation", checkpoint_path)
        .trigger(availableNow=True)
        .toTable("workspace.tp_spark_XYZ.bronze_stream_taxi"))
query.awaitTermination()
duration = time.time() - start

log.append({
    "iteration": i + 1,
    "rate": RATE,
    "duration_seconds": round(duration, 2),
    "rows_total": spark.read.table("workspace.tp_spark_XYZ.bronze_stream_taxi").count()
})
print(f"Itération {i+1}/10 – durée : {duration:.2f}s")
time.sleep(20)

import pandas as pd
print(pd.DataFrame(log).to_string(index=False))

```

Ce tableau de log est le livrable central du challenge de volume. Il contient une ligne par itération, avec la durée de traitement de chaque run. C'est l'équivalent de la Batch Duration en streaming continu.

Lire l'historique dans la Spark UI

La Spark UI reste accessible sur serverless. Dans l'onglet **Structured Streaming**, chaque run `availableNow` apparaît comme une entrée dans l'historique des queries, avec sa durée, le nombre de lignes traitées, et les offsets lus.

Ce que vous pouvez y lire :

- la durée de chaque run (cohérente avec votre `time.time()`),
- le nombre de lignes traitées par run,
- les erreurs éventuelles.

Ce que vous ne verrez pas dans votre TP: Input Rate, Process Rate et Trigger Gap — ces métriques n'ont de sens que sur un job continu avec `processingTime` .

Sur un vrai cluster avec `processingTime` , ces métriques s'interprètent ainsi:

- **Input Rate** est le nombre de lignes reçues par seconde, calculé en moyenne sur les derniers micro-batches. C'est la vitesse à laquelle la source produit des données. Si cette valeur monte progressivement, le producteur accélère.
- **Process Rate** est le nombre de lignes traitées par seconde par le pipeline. Si `Process Rate < Input Rate` , le pipeline ne suit plus : les données s'accumulent dans la source plus vite qu'elles ne sont consommées. C'est le premier signal de saturation.
- **Batch Duration** est le temps de traitement de chaque micro-batch. Si cette valeur dépasse le trigger interval (par exemple 35 secondes pour un trigger de 30 secondes), Spark n'a pas le temps de finir un batch avant que le suivant commence. Les batches s'accumulent en file et le retard croît indéfiniment.
- **Trigger Gap** est le temps d'attente entre la fin d'un batch et le démarrage du suivant. Sur un pipeline sain, ce gap est positif : Spark attend le prochain trigger. Quand le Batch Duration dépasse le trigger interval, le Trigger Gap tombe à zéro : Spark redémarre immédiatement, signe que le pipeline est en retard structurel.

Ces quatre métriques donnent une image complète de l'état de santé d'un pipeline streaming en production. Dans le contexte de ce cours avec `availableNow` , c'est le tableau de log de la boucle qui joue ce rôle.

Ce qu'on cherche à observer

Avec la boucle, le point de saturation se détecte différemment du streaming continu. Il n'y a pas de batches qui s'accumulent à l'infini. En revanche, la durée d'un run peut dépasser le `time.sleep(20)` : les fichiers Parquet générés par le simulateur s'accumulent dans le Volume plus vite qu'un run ne les traite. L'itération suivante du simulateur écrit un nouveau fichier avant que le run précédent ne soit terminé. Le prochain run traite alors deux fichiers au lieu d'un, et ainsi de suite.

Le point de saturation est donc le `rate` à partir duquel la durée d'un run dépasse 20 secondes (le `time.sleep`). Au-delà, les fichiers s'accumulent et la charge de chaque run croît à chaque itération.

2. Le challenge du volume : l'épreuve du feu

Jusqu'à maintenant, vous avez travaillé avec `rate=20` (20 trajets par batch). C'est très confortable. Un système en production gère des centaines à des milliers d'événements par seconde.

Sur compte Databricks gratuit, vous ne pouvez pas aller aussi loin. Mais vous pouvez trouver votre point de rupture : le `rate` auquel votre pipeline commence à saturer.

C'est le **challenge de volume** de ce cours.



Sur serverless Databricks, il n'y a pas de trigger continu ni de trigger interval au sens classique. On utilise `availableNow=True` dans une boucle Python. La Batch Duration ne se lit donc pas comme une métrique en direct dans la Spark UI — elle se mesure comme le temps d'exécution d'un run `availableNow`, calculé dans la boucle avec `time.time()` avant et après `query.awaitTermination()`.

La courbe latence vs throughput reste identique à ce qui est demandé dans le TP : pour chaque rate, vous mesurez la durée moyenne d'un run `availableNow` et vous tracez la courbe. Le point de saturation est le rate à partir duquel cette durée dépasse le temps que vous vous êtes donné entre deux itérations (le `time.sleep` de la boucle). La boucle de mesure est la même que celle introduite en Séance 1, avec un `time.time()` encadrant `query.awaitTermination()`.

3. Optimisations applicables sur compte gratuit

Quand vous approchez de la saturation, plusieurs leviers existent. Attention : certains sont inefficaces ou contre-productifs sur un single-node.

Ce qui fonctionne :

- **Augmenter l'interval de trigger.** Un trigger de 60 secondes donne deux fois plus de temps pour traiter chaque batch. Le débit instantané reste identique, mais le pipeline est moins sensible aux pics courts.
- **Réduire les transformations coûteuses par batch.** Un Stream-Stream Join dans chaque micro-batch est coûteux. Le passer en Stream-Static Join divise souvent par 3 le temps de batch.
- **Mettre en cache les tables statiques.** La table de zones, les stats batch, les labels de clusters : ce sont des données qui ne changent pas entre les batches. Les charger à chaque batch est un gaspillage.

```
# Charger une fois, mettre en cache
df_zones = spark.read.table("workspace.tp_spark_XYZ.ref_taxi_zones").cache()
df_zones.count() # Force le cache
print("Table zones en cache")
```

- **Optimiser la table Delta cible.** Après des heures de streaming, la table Delta accumule des petits fichiers (un par micro-batch). `OPTIMIZE` + `ZORDER` (Spark Core S5) s'appliquent ici aussi.

```
OPTIMIZE workspace.tp_spark_XYZ.gold_stream_taxi
ZORDER BY (tpep_pickup_datetime, PULocationID)
```

Ce qui ne fonctionne pas sur single-node :

- **Augmenter le nombre de partitions Spark.** Sur un single-node, vous avez de toute façon un seul executor. `spark.conf.set("spark.sql.shuffle.partitions", "200")` ne sert à rien, voire ralentit (overhead de coordination).
- **Ajouter des workers.** Le compte gratuit ne le permet pas.

4. Arrêt gracieux et reprise

Un job streaming en production ne s'arrête pas avec Ctrl+C. Il y a un protocole.

```
# Arrêt propre : attend la fin du micro-batch en cours
query.stop()

# Vérification
print(f"Job actif : {query.isActive}")
print(f"Status : {query.status}")
```

Si le cluster est tué brutalement (timeout Databricks), le checkpoint contient l'état du dernier batch complété. Au redémarrage, Spark reprend depuis ce point. Les

données en cours de traitement au moment du crash sont rejouées (d'où l'importance de l'Exactly-Once de Delta).



Sur serverless Databricks avec `availableNow=True`, le job s'arrête tout seul après avoir traité ce qui est disponible. `query.stop()` n'est pas nécessaire et `query.isActive` retourne déjà `False` dès que `awaitTermination()` rend la main. L'arrêt gracieux se gère naturellement via la boucle Python : chaque itération attend la fin du run avant de passer à la suivante. En cas de crash du cluster, le comportement est identique au streaming continu : le checkpoint préserve l'état et le prochain run repart là où il s'était arrêté.

5. Bilan batch vs streaming

Après cinq séances, vous êtes en position de comparer les deux paradigmes de manière informée.

Dimension	Batch (Spark Core)	Streaming (ce cours)
Latence	Minutes à heures	Secondes à minutes
Complexité	Faible	Élevée (watermarks, checkpoints, modes de sortie)
Débogage	Simple (job fini, on regarde les résultats)	Difficile (job tourne, logs à lire en direct)
Coût opérationnel	Job ponctuel	Process continu 24/7
Cas d'usage	Reporting, entraînement ML, analyses profondes	Alertes, monitoring, scoring temps réel
Garanties qualité	Facile à auditer	Quarantaine obligatoire, test de résilience nécessaire

La conclusion n'est pas "le streaming est mieux". C'est "le streaming est nécessaire pour les cas où la latence compte, et évitable pour les autres".

TP Séance 5 : Challenge de volume et livrable final

Ce qui vous est donné

Les rates à tester : 20, 50, 100, 200 trajets par batch. Le protocole est identique pour tous : 10 batches par rate, Batch Duration relevée depuis la Spark UI, arrêt propre entre chaque test. Si le cluster crashe à rate=200, documentez-le et passez à la suite.

Ce que vous devez produire

Partie A : challenge de volume.

Testez les quatre rates suivants : 20, 50, 100, 200. Pour chaque rate, relancez la boucle de 10 itérations avec le rate correspondant. À chaque itération, mesurez la durée du run avec `time.time()` (voir le code de la section 1). Remplissez un tableau de log : rate testé, durée moyenne par run, durée maximale par run, nombre de lignes cumulées, observations.

Si à un rate donné la durée d'un run dépasse 20 secondes (le `time.sleep`), notez-le : c'est le signal de saturation. Si le cluster crashe, documentez-le et passez au rate suivant.

Tracez la courbe : sur l'axe X les rates testés, sur l'axe Y la durée moyenne d'un run. Ajoutez une ligne horizontale à 20 secondes (la valeur du `time.sleep`). Le point où la courbe croise cette ligne est votre point de saturation.

Partie B : optimisation.

Choisissez au moins deux optimisations parmi : augmenter le trigger interval, mettre en cache les tables statiques, remplacer un Stream-Stream Join par un Stream-Static Join si applicable, appliquer `OPTIMIZE ZORDER` sur la table Gold. Pour chaque optimisation, mesurez la Batch Duration avant et après. Si l'amélioration est faible ou nulle, expliquez pourquoi.

Partie C : rapport final.

Un diagramme textuel ou image montrant les tables, les jobs, les checkpoints et les flux de données du pipeline complet.

Les mêmes KPI que Spark Core S3 (montant moyen par borough avec intervalle de confiance) calculés sur vos données streaming. Les distributions sont-elles cohérentes ? Si non, pourquoi ?

Une section d'au moins 10 lignes intitulée "Ce qui ne fonctionne pas" : simplifications faites par rapport à un système de production réel, limites observées, points qu'on ferait différemment avec plus de ressources.

Livrable final : Notebook **S5_Final_TRIGRAMME**

Le notebook `.dbc` exporté contient les trois parties. Il sera évalué comme un tout.

Grille d'évaluation finale (Séance 5 + ensemble du module)

Critère	Pondération	Insuffisant	Attendu	Excellent
Challenge volume	25%	Un seul rate testé	3-4 rates + courbe avec ligne de saturation à 20s	Courbe + point de saturation identifié + tableau de log complet
Optimisations	20%	Aucune	2 optimisations mesurées (durée de run avant/après)	2+ avec analyse de l'effet observé et explication si effet faible
Comparaison Spark Core	20%	Absente	KPI comparés	• explication des écarts
Synthèse pipeline	15%	Absente	Diagramme lisible	• description du flux de données
Honnêteté technique	10%	Absente	Problèmes mentionnés	• simplifications analysées
Qualité du code	10%	Code non documenté	Commentaires et cellules Markdown	Structuré, reproductible, Markdown clair

Les notebooks qui tournent mais qui ne documentent pas seront plafonnés à la mention Passable.